

Robotics Laboratory 2025-2026: Short Report

Leonardo Luigi Pepe, Lorenzo Tomai

May 2026

TurtleBot number : 01

1 Setup description and experiments preparation

The experimental activity was carried out on a TurtleBot mobile robot modeled as a unicycle system with differential drive kinematics. The robot is equipped with two independently actuated wheels and onboard sensors including wheel encoders and an IMU. External position measurements were provided by the motion capture system available in the laboratory.

Communication with the robot was handled through MATLAB/Simulink, which was used both to send control inputs and to acquire sensor data in real time. The overall setup allowed a direct comparison between simulated and experimental results within a unified framework.

The robot state is defined as $q = [x, y, \theta]^T$, where x and y represent the position in the plane and θ is the heading angle with respect to the global reference frame. This state is estimated through sensor fusion and used for both control and localization purposes.

The nominal parameters provided for the robot were:

- Wheel radius: $r = 0.03m$
- Wheel distance: $d = 0.165m$
- Maximum wheel angular velocity: $\omega_M = 10rad/s$
- Sampling time: $T_s = 0.04s$

We would like to highlight an important aspect observed during the analysis of the sensor data, in particular regarding the IMU gyroscope.

The angular velocity measured by the IMU presented two main issues. First, the z axis was inverted with respect to the motion capture reference frame, which is instead consistent with the robot reference frame. Second, the measured angular velocity showed a significant discrepancy compared to the value obtained by differentiating the orientation angle θ of both the desired trajectory and the pose measured by the motion capture system.

After a careful analysis, it was found that, in addition to the expected conversion from degrees per second to radians per second, an additional scaling factor of 0.1¹ had to be applied to the value returned by the sensor.

After introducing these corrections, the IMU data became consistent with the other available measurements and could be reliably used for state estimation, as discussed in Task 2. These corrections were implemented directly inside the Simulink model so that the collected data could also be used for online (real-time) analysis.²

2 Task 1: Sim-to-real gap and identification

2.1 Task Description

This task bridges the gap between theory and the practical implementation of the DDR model. Previous simulations assumed an ideal environment characterized by:

- ideal actuators,
- absence of wheel slip,
- perfectly known kinematic parameters (r and d),
- and continuously known robot configuration.

However, practical implementations rely on digital controllers, replacing continuous data with discrete measurements sampled every T_s seconds.

Furthermore, accurately identifying the real parameters is crucial. Since planned trajectories and feedforward velocities depend heavily on these values, even minor sim-to-real discrepancies can significantly degrade performance. Consequently, this section first compares real-time execution with offline simulations, and then discusses the parameter identification process.

¹Even if the real factor is 0.125, the results presented in this report are based on the initial rough estimate of 0.1.

²The original uncorrected sensor data were still saved separately in the scripts for completeness and further analysis.

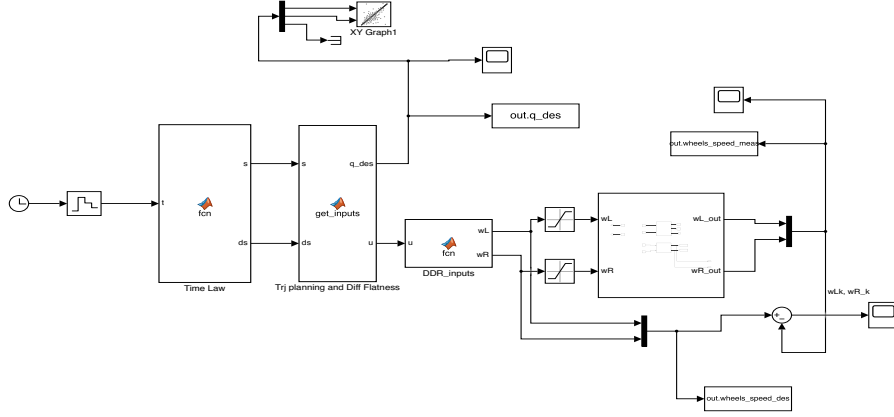


Figure 1: Task 1 Simulink scheme

2.2 Adopted Strategy

This task transitions from ideal simulations to a practical DDR implementation, addressing digital sampling ($T_s = 0.04$ s), sensor noise, and parameter uncertainties. We employ an 8-shaped trajectory with a trapezoidal velocity profile (v_c, a). Trajectory generation and flat-output computation are executed online via Simulink. The geometric velocities $\tilde{v}, \tilde{\omega}$ are analytically derived from the path equations and scaled by the time derivative $\dot{s}(t)$ to compute the final feedforward commands $v(t)$ and $\omega(t)$.

The trapezoidal time law is defined by a cruise velocity v_c and an acceleration a , computed as follows to ensure the total path length is normalized to $s = 1$:

$$v_c = \frac{1}{T_a + T_c}, \quad a = \frac{v_c}{T_a}$$

The position $s(t)$ along the path is described by the following piecewise function:

$$s(t) = \begin{cases} \frac{1}{2}at^2 & \text{if } 0 \leq t < T_a \\ \frac{1}{2}aT_a^2 + v_c(t - T_a) & \text{if } T_a \leq t < T_a + T_c \\ \frac{1}{2}aT_a^2 + v_cT_c + v_c(t - T_a - T_c) - \frac{1}{2}a(t - T_a - T_c)^2 & \text{if } T_a + T_c \leq t < 2T_a + T_c \\ 1 & \text{if } t \geq 2T_a + T_c \end{cases}$$

Accordingly, the path derivative $\dot{s}(t)$ is:

$$\dot{s}(t) = \begin{cases} at & \text{if } 0 \leq t < T_a \\ v_c & \text{if } T_a \leq t < T_a + T_c \\ v_c - a(t - T_a - T_c) & \text{if } T_a + T_c \leq t < 2T_a + T_c \\ 0 & \text{if } t \geq 2T_a + T_c \end{cases}$$

The eight-shaped trajectory is defined in the Cartesian space parameterized by the path variable s . Given a scaling radius R and a spatial frequency ω_{trj} , the desired position is formulated as follows:

$$x(s) = R \sin(2\omega_{trj}s), \quad y(s) = R \sin(\omega_{trj}s) \quad (1)$$

By analytically computing the first and second spatial derivatives ($x'(s), y'(s)$ and $x''(s), y''(s)$), we can define the full robot state. Exploiting the differential flatness of the unicycle model, the desired heading θ_{des} and the geometric feedforward velocities ($\tilde{v}$ and $\tilde{\omega}$) are entirely determined by the path's geometry:

$$\theta_{des} = \text{atan2}(y'(s), x'(s)) \quad (2)$$

$$\tilde{v} = \sqrt{x'(s)^2 + y'(s)^2}, \quad \tilde{\omega} = \frac{x'(s)y''(s) - y'(s)x''(s)}{x'(s)^2 + y'(s)^2} \quad (3)$$

Finally, to obtain the actual time-dependent feedforward linear and angular velocities (v and ω) applied to the robot, these geometric terms must be scaled by the path derivative $\dot{s}(t)$ provided by the time law:

$$v(t) = \tilde{v} \cdot \dot{s}(t), \quad \omega(t) = \tilde{\omega} \cdot \dot{s}(t) \quad (4)$$

2.3 Simulink Implementation & Experimental Setup

The Simulink model (Fig. 1) computes the desired trajectory online, mapping the feedforward vector $u = [v, \omega]^T$ to the left and right wheel speeds. These commands are saturated at $\pm 10 \text{ rad s}^{-1}$ and sent to the TurtleBot via ROS. Real-time feedback (wheel velocities, IMU, MoCap) is logged for offline analysis. The gyroscope signal is preprocessed via sign inversion, conversion to radians, and a 0.1 rad s^{-1} factor. We tested three time constants ($T_c = 18, 30, 45$); as predicted, larger T_c values yield slower reference velocities.

2.4 Parameter Identification

Methodology

The kinematic identification procedure aims to accurately estimate the fundamental parameters of the Differential Drive Robot (r and d) while compensating for MoCap systematic calibration errors. The algorithm processes wheel velocities and MoCap poses sampled at $T_s = 0.04 \text{ s}$. Initially, MoCap data is

preprocessed by inverting the Y and orientation axes to match the standard unicycle frame and unwrapping angles to prevent 2π discontinuities.

The process is split into two main phases. First, an offset-free identification uses an Euler integration approximation, expressing the one-step pose variation $Y = q_{k+1} - q_k$ as a linear regression $Y = \Phi \tilde{w}$, where $\tilde{w} = [r, r/d]^T$. A constrained least-squares solver is applied to guarantee physically meaningful positive values ($r, d \geq 0$).

Second, physical misalignments between the MoCap tracking markers and the robot’s actual center are modeled via local offsets $(x^{off}, y^{off}, \theta^{off})$. This formulation yields an extended linear parameter vector $\tilde{w}_{cal} = [r, r/d, rx^{off}/d, ry^{off}/d]^T$, although the equations remain highly nonlinear with respect to θ^{off} . To solve this mixed problem, a nested optimization is deployed: a nonlinear unconstrained search iteratively minimizes the tracking error to find the optimal θ^{off} , while the Cartesian offsets and physical parameters are computed via linear least-squares at each step. Finally, reversing these estimated offsets accurately calibrates the raw MoCap trajectory.

Identified Parameters: $T_c = 18$

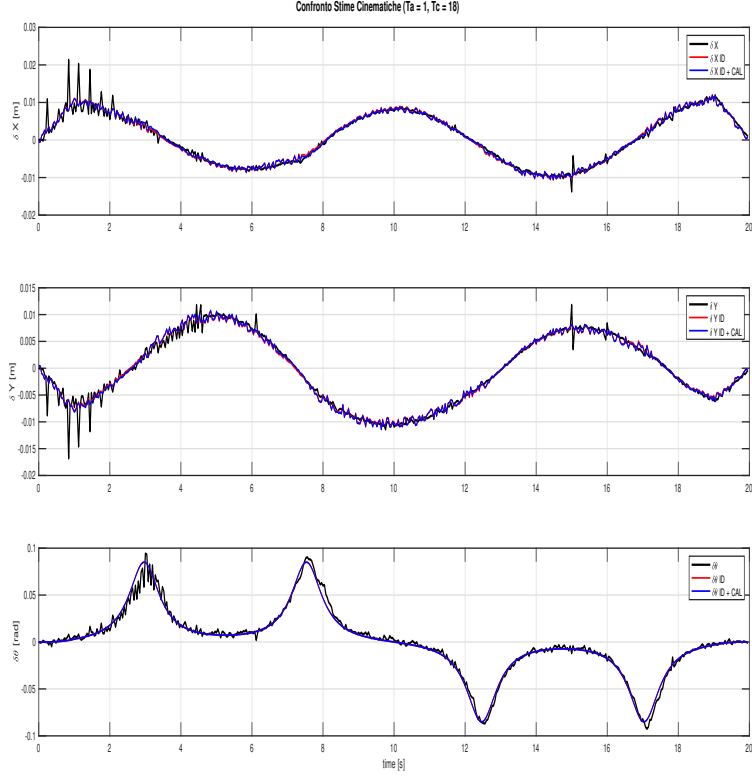


Figure 2: $T_c = 18$

Table 1: Comparison of kinematic parameters r and d ($T_c = 18$).

Parameter	Unconstrained	Constrained	Calibration
r [m]	0.033 50	0.033 50	0.033 50
d [m]	0.166 93	0.166 93	0.166 94

Table 2: Calibration offsets and error statistics for (x, y, θ) states ($T_c = 18$).

State	Offset	Mean Error	Std Dev	Unit
x	-0.003 46	3.68×10^{-5}	1.30×10^{-3}	[m]
y	-0.004 31	-4.70×10^{-5}	1.11×10^{-3}	[m]
θ	0.019 50	3.78×10^{-4}	4.65×10^{-3}	[rad]

Identified Parameters: $T_c = 30$

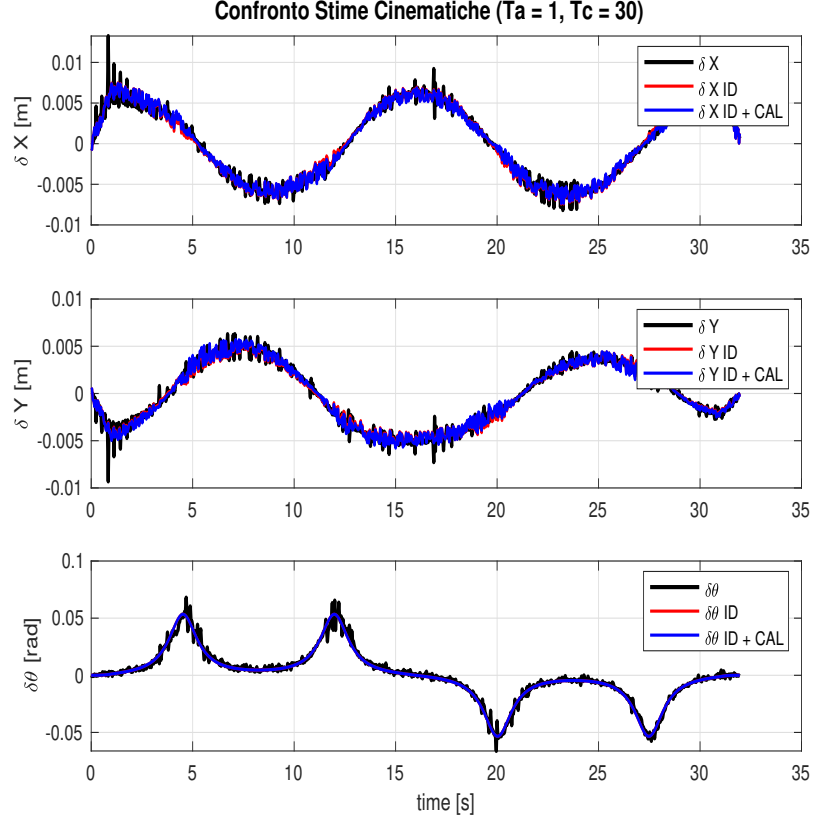


Figure 3: $T_c = 30$

Table 3: Comparison of kinematic parameters r and d ($T_c = 30$).

Parameter	Unconstrained	Constrained	Calibration
r [m]	0.032 93	0.032 93	0.032 93
d [m]	0.160 37	0.160 37	0.160 40

Table 4: Calibration offsets and error statistics for (x, y, θ) states ($T_c = 30$).

State	Offset	Mean Error	Std Dev	Unit
x	-0.003 84	-9.32×10^{-6}	9.19×10^{-4}	[m]
y	-0.008 58	-1.24×10^{-5}	7.09×10^{-4}	[m]
θ	0.016 38	4.13×10^{-4}	3.13×10^{-3}	[rad]

Identified Parameters: $T_c = 45$

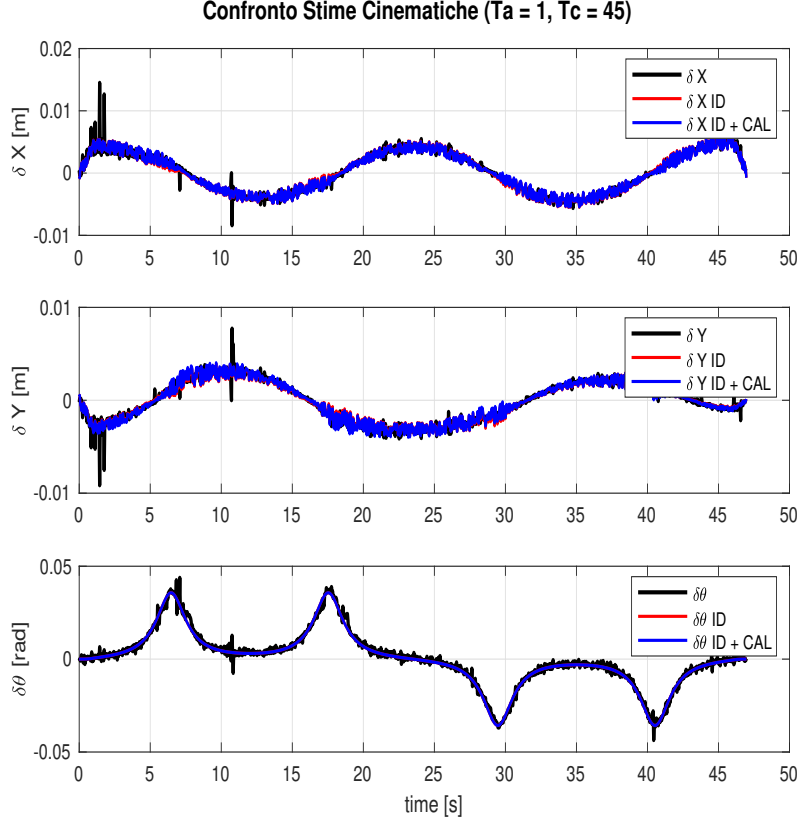


Figure 4: $T_c = 45$

Table 5: Comparison of kinematic parameters r and d ($T_c = 45$).

Parameter	Unconstrained	Constrained	Calibration
r [m]	0.032 44	0.032 44	0.032 45
d [m]	0.159 18	0.159 18	0.159 24

Table 6: Calibration offsets and error statistics for (x, y, θ) states ($T_c = 45$).

State	Offset	Mean Error	Std Dev	Unit
x	-0.004 16	1.69×10^{-5}	8.28×10^{-4}	[m]
y	-0.009 66	-2.29×10^{-5}	6.01×10^{-4}	[m]
θ	0.017 69	3.58×10^{-4}	1.79×10^{-3}	[rad]

The identification results demonstrate model consistency, as unconstrained and constrained solvers yielded identical r and d . However, parameters vary with trajectory speed: at $T_c = 18$, aggressive maneuvers cause micro-slip and actuator saturation, artificially inflating the estimated dimensions. Conversely, at $T_c = 45$, low speeds degrade the Signal-to-Noise Ratio (SNR). Thus, $T_c = 30$ ($r \approx 0.0329$ m, $d \approx 0.1604$ m) is selected as the nominal model, offering the optimal balance between dynamic excitation and noise robustness.

2.5 Sim-to-Real Gap and System Comparison

Simulated vs. Actual System: $T_c = 18$

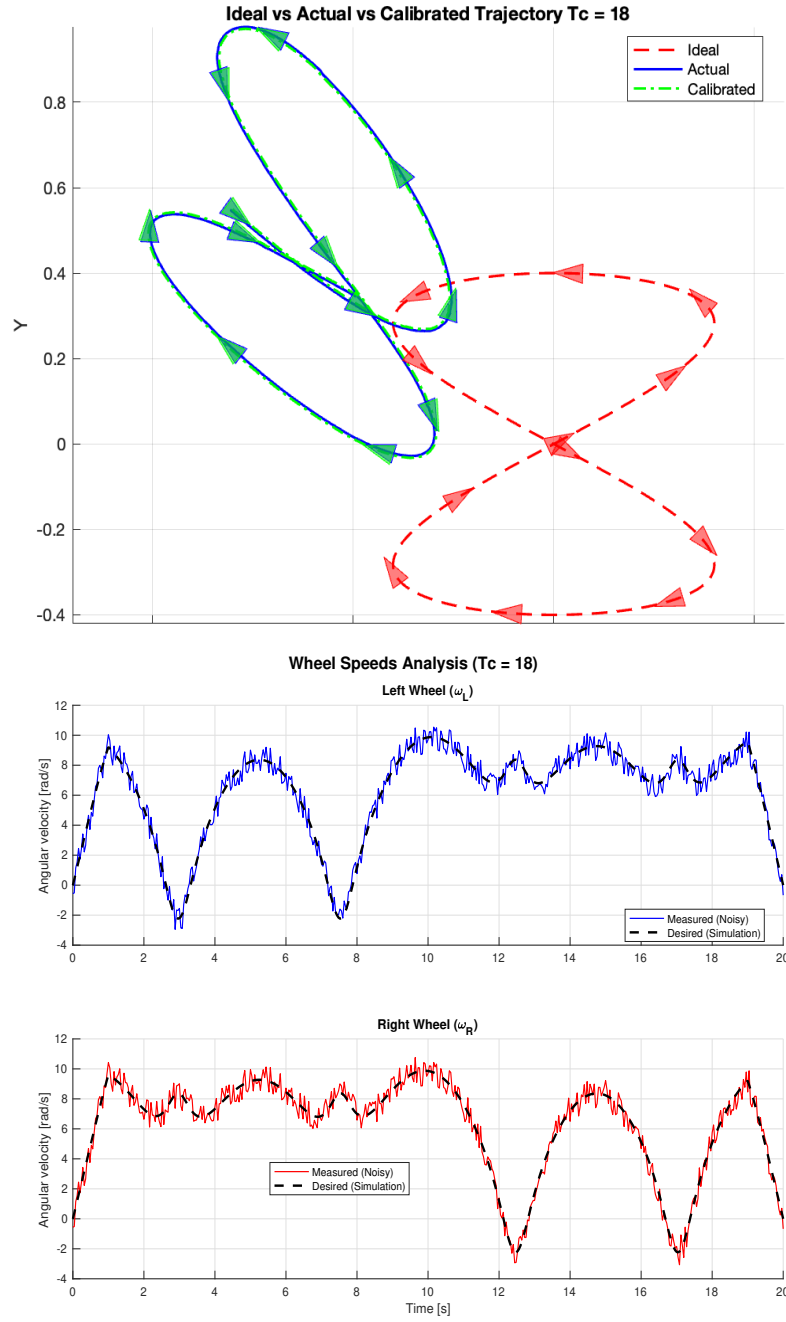


Figure 6: $T_c = 18$

Simulated vs. Actual System: $T_c = 30$

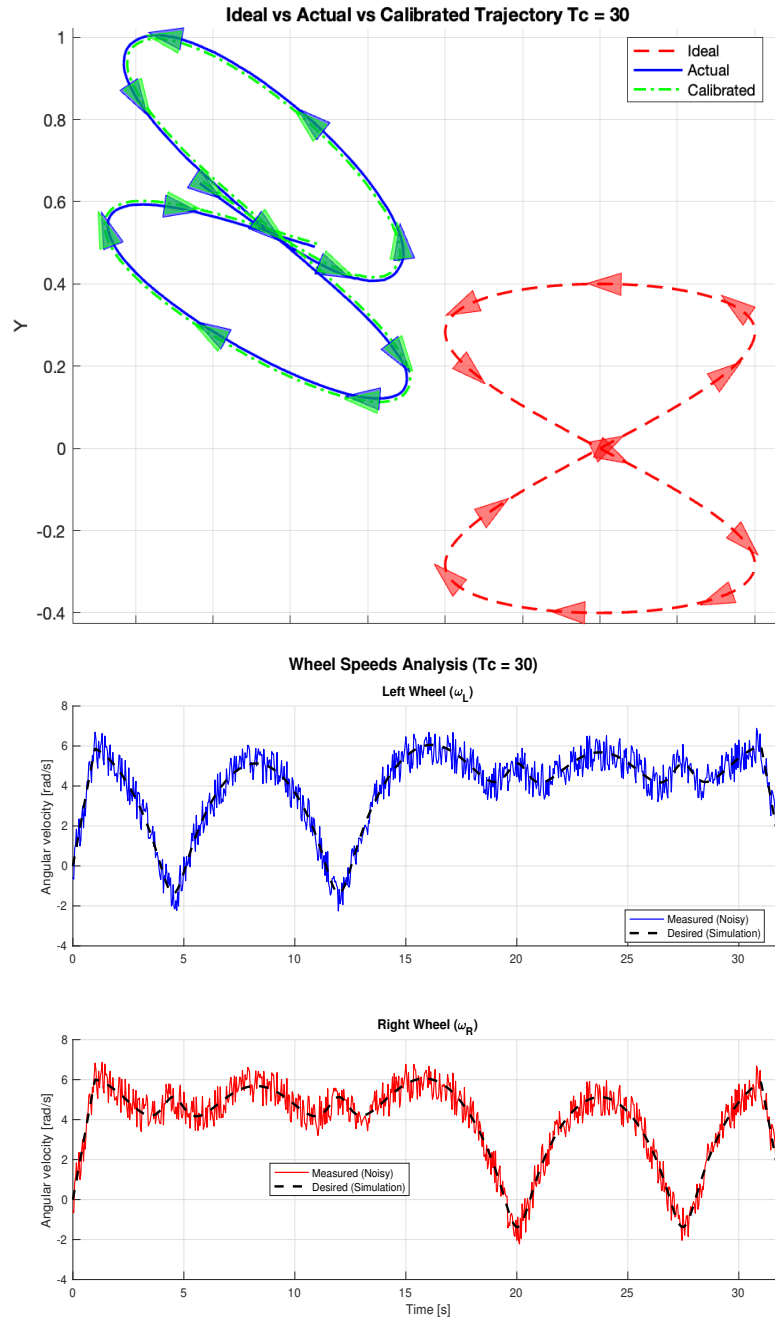


Figure 8: $T_c = 30$

Simulated vs. Actual System: $T_c = 45$

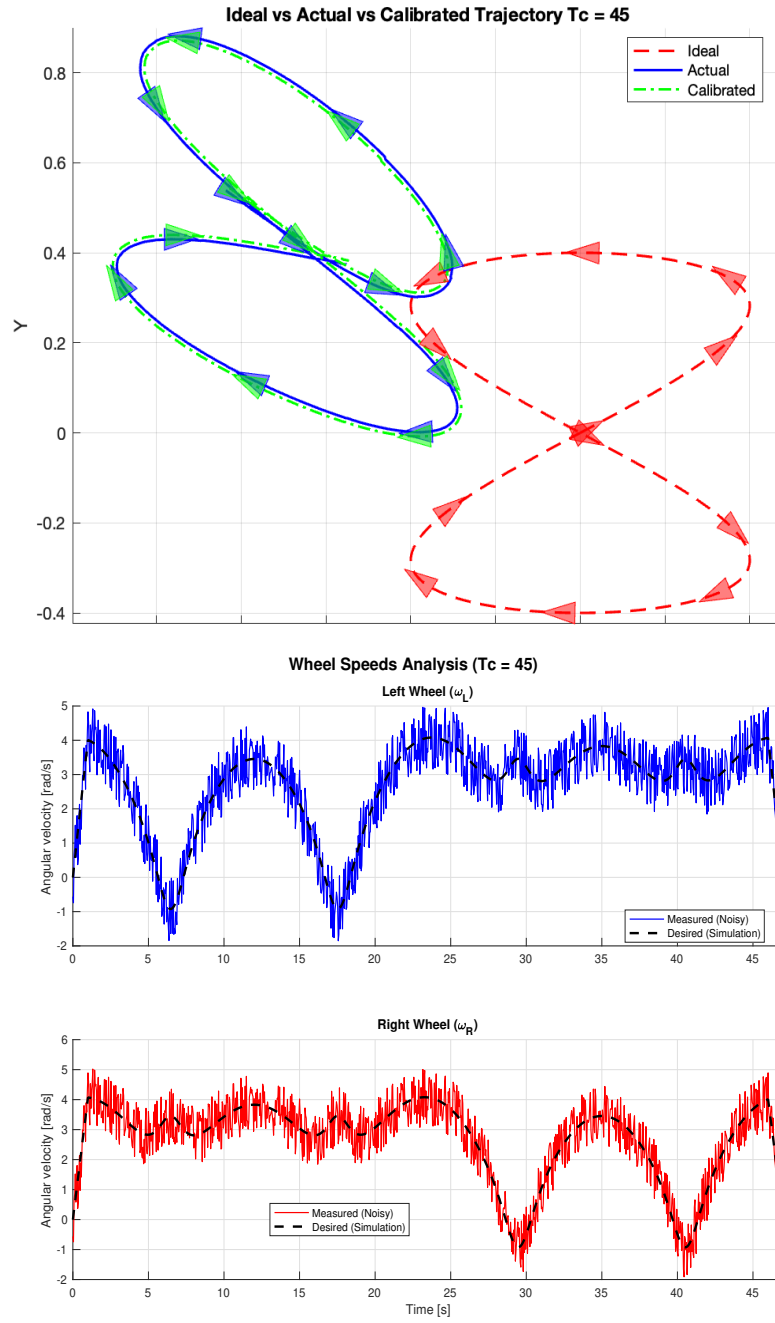


Figure 10: $T_c = 45$

Simulated vs. Actual System: Comparison and Analysis

Experimental trajectories exhibit spatial drift (shift and rotation) relative to the desired path. This is expected, as the robot relies on open-loop feedforward control without MoCap or Kalman feedback regulation.

Table 7: Wheel speed boundaries and noise analysis for different time constants T_c .

T_c	Noise Std [rad s ⁻¹]	Min Des [rad s ⁻¹]	Max Des [rad s ⁻¹]	Min Meas [rad s ⁻¹]	Max Meas [rad s ⁻¹]	Rel. Noise [%]
18	0.5644	-2.2332	9.8766	-3.0648	10.7563	5.71
30	0.5640	-1.3693	6.0538	-2.2210	6.8640	9.32
45	0.5631	-0.9227	4.0797	-1.9088	5.0202	13.8

Table 7 highlights a critical SNR degradation. Absolute sensor noise remains constant ($\approx 0.56 \text{ rad s}^{-1}$), but as T_c increases, the reference velocity shrinks. Consequently, relative noise surges from 5.71 % ($T_c = 18$) to 13.80 % ($T_c = 45$), heavily impacting the smoothness of slower motions. Finally, at $T_c = 18$, measured velocities peak at 10.76 rad s^{-1} , exceeding the 10 rad s^{-1} software saturation. This is directly caused by the additive sensor noise and motor overshoot during rapid accelerations.

3 Task 2: EKF Localization

3.1 Task Description

The goal of this task was to evaluate and tune the Extended Kalman Filter (EKF) for robot localization using different sensor configurations.

The following scenarios were analyzed:

- only encoder
- encoder + IMU
- encoder + IMU + motion capture with $p_{loss} = 0.9$
- encoder + IMU + motion capture with $p_{loss} = 0.99$

The tuning focused on process covariance and measurement noise covariance matrices.

3.2 Adopted Strategy

The EKF was implemented to improve robot localization by combining the prediction obtained from the kinematic model with corrections provided by the available sensors. Encoder-only integration accumulates drift over time due

to wheel slip and parameter mismatch. Therefore, correction terms were introduced using IMU and motion capture measurements. The IMU mainly improves the estimation of the heading angle θ , while the motion capture system provides absolute pose measurements for the full state.

The EKF state extends the robot pose with wheel measurements:

$$\hat{\mathbf{z}}_{k+1} = [x_{k+1}, y_{k+1}, \theta_{k+1}, \Delta\phi_{L,k}, \Delta\phi_{R,k}, \omega_{L,k+1}, \omega_{R,k+1}]^T$$

The prediction model is based on unicycle kinematics:

$$x_{k+1} = x_k + v_k \cos \theta_k T_s \quad (5)$$

$$y_{k+1} = y_k + v_k \sin \theta_k T_s \quad (6)$$

$$\theta_{k+1} = \theta_k + \omega_k T_s \quad (7)$$

The EKF follows the standard prediction-correction structure:

$$\text{Prediction: } \hat{\mathbf{z}}_{k|k-1} = f(\hat{\mathbf{z}}_{k-1}, \mathbf{u}_k) \quad (8)$$

$$P_{k|k-1} = F_k P_{k-1} F_k^T + D \quad (9)$$

$$\text{Correction: } K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1} \quad (10)$$

$$\hat{\mathbf{z}}_k = \hat{\mathbf{z}}_{k|k-1} + K_k (\mathbf{y}_k - H \hat{\mathbf{z}}_{k|k-1}) \quad (11)$$

$$P_k = (I - K_k H) P_{k|k-1} \quad (12)$$

The tuning process focused on the covariance matrices D and R . The process covariance matrix D represents uncertainty in the model prediction: higher values increase the filter reliance on measurements. The measurement covariance matrix R represents sensor uncertainty: higher values reduce the correction effect of measurements.

The tuning affects the confidence associated with the predicted state and consequently the size of the uncertainty region described by the covariance matrix P .

3.3 Simulink Implementation

To test the Kalman localization technique, we used the same data from the previous task, including the calibration component performed offline.

A single Simulink model was developed, featuring a generic Kalman estimation block suitable for all considered scenarios. The inputs consist of the reference control input u , given by the desired wheel velocities, and a combined measurement vector including encoder wheel displacements, IMU gyroscope angular velocity, and motion capture state.

The motion capture input is affected by a data loss logic, which simulates missing measurements by setting the state to NaN with a probability p_{loss} .

Inside the Kalman estimation block, a full observation matrix H is defined. To select which measurements are effectively used in the estimation, very high values of noise variance (10^9) are assigned to the corresponding entries in the covariance matrix, effectively excluding them from the correction step.³

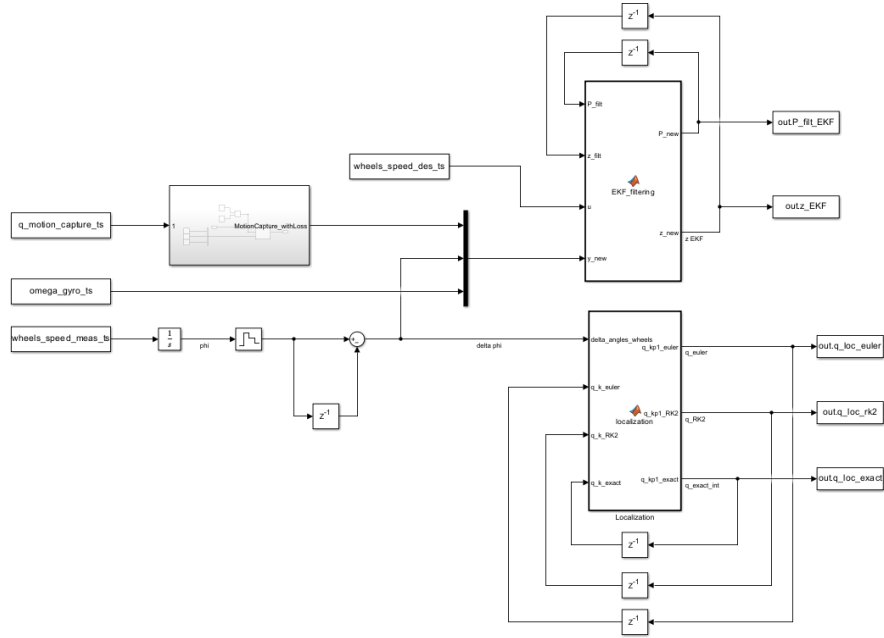


Figure 11: Task 2 Simulink scheme

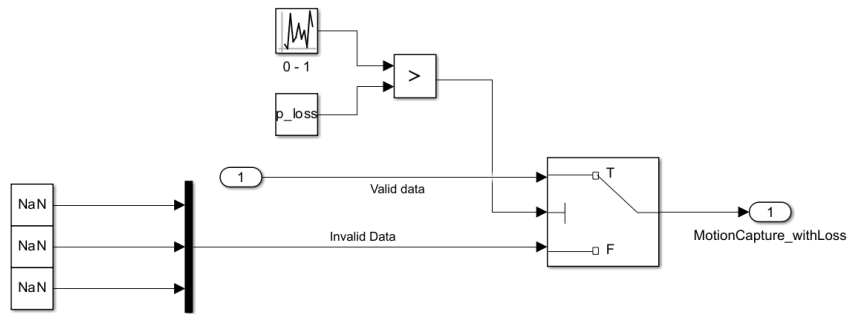


Figure 12: p_loss Simulink-block scheme

To simplify the tuning process, additional analysis functions were implemented. These tools graphically show the estimation uncertainty associated with a given covariance value. In particular, functions were developed to:

- compare the estimation error with respect to the motion capture measurements and verify whether it remains inside the 3σ confidence bounds;
- plot covariance ellipses along the trajectory in order to verify that the actual robot position remains inside the estimated uncertainty region.

3.4 Results

Several numerical tests were performed on the covariance matrices R and D . As expected from the EKF theory, increasing the values of the process covariance matrix D (in particular the variances associated with x , y , and θ) injects additional uncertainty into the prediction model at each iteration.

The values of R were initially fixed to representative values based on sensor characteristics and empirical observations:

$$\sigma_{\text{enc}} = \frac{q}{\sqrt{12}} \quad \sigma_{\text{imu}} = 0.01 \quad \sigma_{\text{mot_cap}} = 0.001$$

With the encoder quantization $q = \frac{2\pi}{4096}$. The corresponding measurement covariance matrix is defined as:

$$R = \text{diag} \left([\sigma_{\text{mot_cap}}, \sigma_{\text{mot_cap}}, \sigma_{\text{mot_cap}}, \sigma_{\text{enc}}, \sigma_{\text{enc}}, \sigma_{\text{imu}}]^2 \right)$$

The encoder variance is just an estimate from theory, the motion capture variance is intentionally kept low due to the high accuracy of the system while the IMU variance is set to a relatively high value because sensor measurements seem inaccurate and poorly calibrated.

The tuning process initially focused on the *encoder-only* configuration. Using very small values for the process covariance matrix D produced unrealistically low covariance values, while the actual odometry error observed experimentally was significantly larger. In this condition, the filter relied excessively on the kinematic model (see Fig. 13).

³Using a reduced and case-specific observation matrix H may introduce issues due to changes in input dimension during online execution.

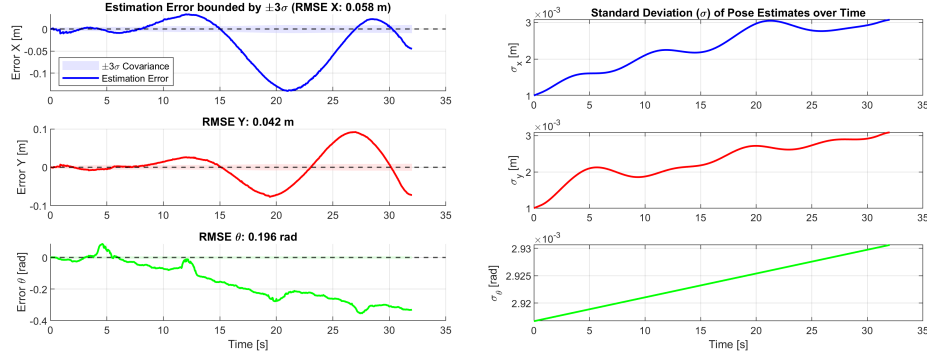


Figure 13: Encoder-only EKF performance with $D = \text{diag}([10^{-4}, 10^{-4}, 10^{-5}, \dots]^2)$

The process covariance matrix D was therefore progressively increased until the uncertainty bounds were consistent with the actual robot deviation. The final tuning produced covariance ellipses that correctly include the effective trajectory error, as shown in Fig. 14.

The ellipses appear elongated more on the lateral direction. This behavior is consistent with the system dynamics, since small heading estimation errors accumulate into significant lateral position deviations, while forward encoder measurements are more accurate.

Adding the IMU measurements, the results were practically identical to the encoder-only configuration. No significant improvement was observed in either the prediction accuracy or the covariance evolution due to the high measurement variance.

On the other hand, when motion capture measurements are included, the covariance rapidly decreases whenever a valid camera measurement is received, and then gradually increases again according to the process covariance D . This behavior is clearer in the case with $p_{loss} = 0.99$ (see Fig 16). However, the initial motion capture covariance appeared slightly too optimistic (the ellipse area didn't include the real trajectory), therefore it was slightly increased for robustness:

$$\sigma_{motion\ capture} = 0.008\ m$$

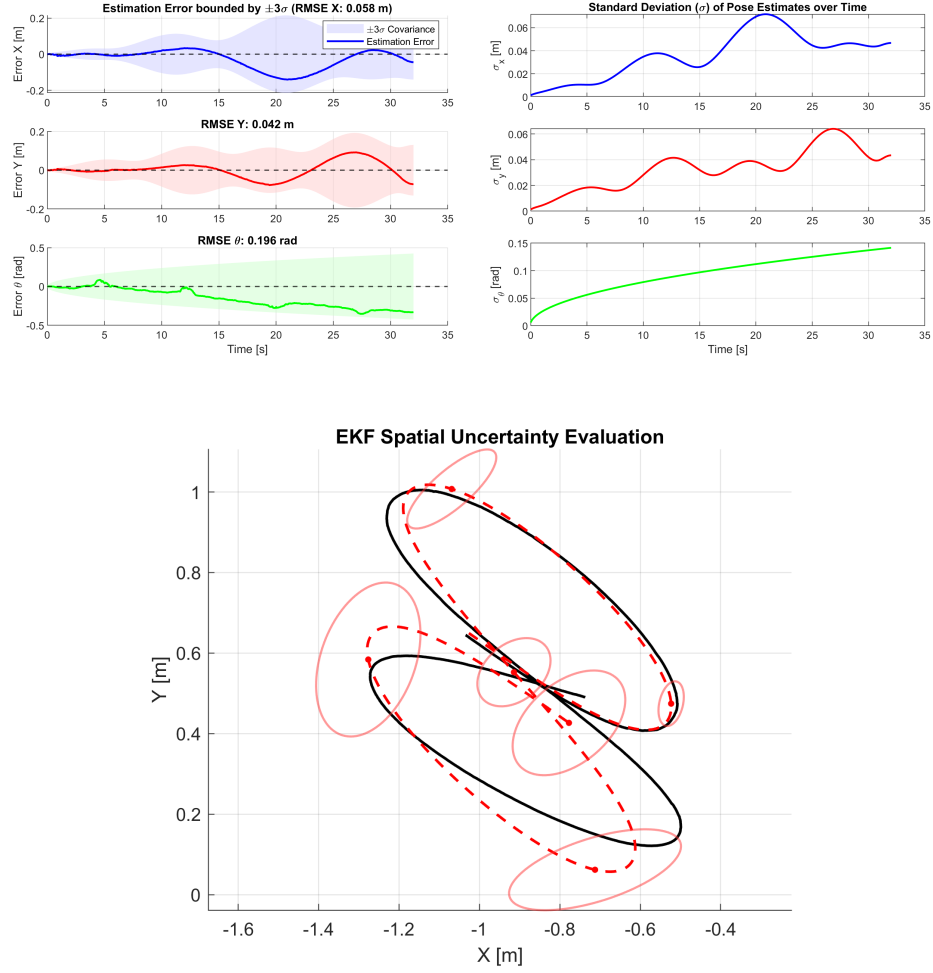


Figure 14: Encoder-only EKF performance with $D = \text{diag}([0.8e-3, 0.8e-3, 5e-3, \dots]^2)$.

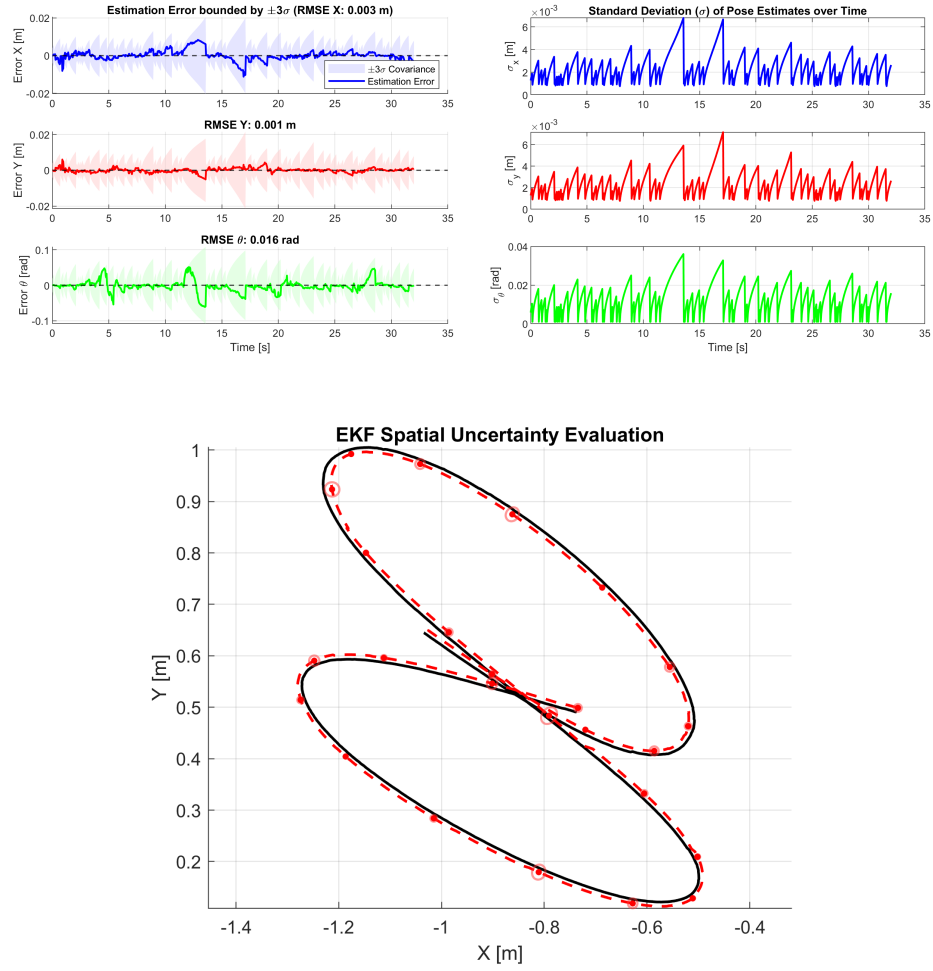


Figure 15: EKF with encoder + IMU + motion capture ($p_{\text{loss}} = 0.90$), tuned D , $\sigma_{\text{mocap}} = 0.001$ m.

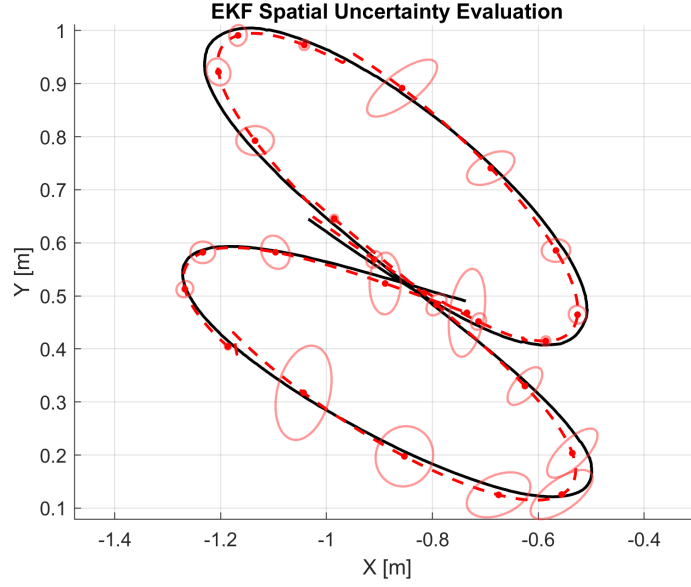


Figure 16: EKF with encoder + IMU + motion capture ($p_{\text{loss}} = 0.99$), tuned D , $\sigma_{\text{mocap}} = 0.001$ m.

To sum up, the final covariance matrices used for the EKF, and corresponding results (Fig. 17), are the following:

$$R = \text{diag} \left([8 \cdot 10^{-3}, 8 \cdot 10^{-3}, 8 \cdot 10^{-3}, \approx 5 \cdot 10^{-4}, \approx 5 \cdot 10^{-4}, 0.01]^2 \right)$$

$$D = \text{diag} \left(\left[0.8 \cdot 10^{-3}, 0.8 \cdot 10^{-3}, 5 \cdot 10^{-3}, \frac{0.0175}{6}, \frac{0.0175}{6}, \frac{0.0175}{6} T_s, \frac{0.0175}{6} T_s \right]^2 \right)$$

3.5 Observations

In general, a small but nearly constant offset can be observed between the Kalman estimate obtained with motion capture always enabled and the actual trajectory measured by the motion capture system itself.

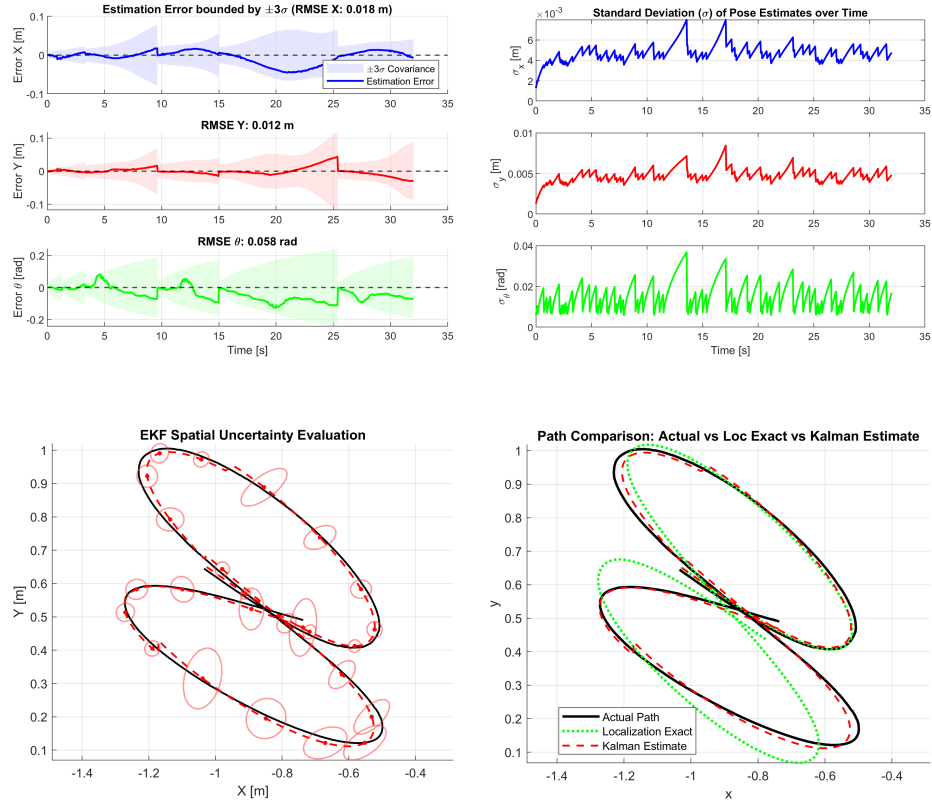


Figure 17: EKF with encoder + IMU + motion capture ($p_{\text{loss}} = 0.99$), tuned D and R .